

ОТЧЕТ ПО ПРОЕКТУ

**Разработка симулятора микроконтроллера STM32F103C8T6 с
Go-сервисом и веб-интерфейсом управления пинами**

ФИО авторов: _____

Руководитель: _____

Оглавление

Введение

1. Техническое задание

2. Архитектура системы

3. Описание реализации

4. Тестирование

Заключение

Список литературы

Приложения

Введение

Настоящий отчет описывает разработку учебного программного комплекса для эмуляции микроконтроллера STM32F103C8T6 (Blue Pill) и управления его состоянием через Go-сервис и веб-интерфейс. Работа выполнена в рамках курса по проектированию вычислительных систем и ориентирована на практическое понимание архитектуры ARM Cortex-M3, модели памяти, MMIO-периферии и отладочного взаимодействия с исполняемой прошивкой.

Ключевой результат текущего этапа — реализованный сквозной сценарий управления пинами процессора: пользователь переключает PA0 во frontend, Go-сервис передает команду в stateful debug bridge, C-симулятор обновляет входной уровень GPIOA, а тестовая прошивка читает GPIOA->IDR и выводит символ H или L через USART1. Такая вертикальная демонстрация показывает, что интерфейс управления влияет на реальное исполнение прошивки, а не только на визуальное состояние UI.

Оформление отчета приближено к структуре выпускной квалификационной работы: титульный лист, оглавление, введение, основная часть, тестирование, заключение, список литературы и приложения. Используются поля и структура, характерные для отчетов по ГОСТ 7.32-2017 и текстовых документов по ГОСТ 2.105-95.

1. Техническое задание

Цель проекта — разработать детерминированный симулятор микроконтроллера STM32F103C8T6 на языке C, а также сервисный и пользовательский слой для запуска, наблюдения и управления эмуляцией. На текущем этапе особый акцент сделан на управлении пинами процессора через Go-сервер и frontend.

Объект разработки — программная модель микроконтроллера с ядром ARM Cortex-M3, памятью Flash/SRAM, шиной, MMIO-устройствами и механизмом прерываний. Предмет разработки — архитектура симуляции, API-контракт сервиса, live/debug session и демонстрационная прошивка, подтверждающая управление входными сигналами.

Таблица 1 — Основные требования к проекту

Требование	Реализация
Загрузка raw firmware	Прошивка загружается во Flash, reset sequence читает MSP и PC из vector table.
Модель CPU	Поддержан MVP-набор Thumb/Thumb-2: арифметика, ветвления, load/store, SVC, PUSH/POP, BL, shifts.
MMIO и периферия	Реализованы TIM2, USART1, NVIC MMIO и минимальная модель GPIOA.
Go-сервис	HTTP API запускает симулятор и создает stateful debug sessions.
Frontend	Статический web UI отображает состояние платы, UART, CPU/peripherals и управляет PA0.
Тестовая программа	Demo firmware читает GPIOA->IDR и отправляет H/L в USART1.

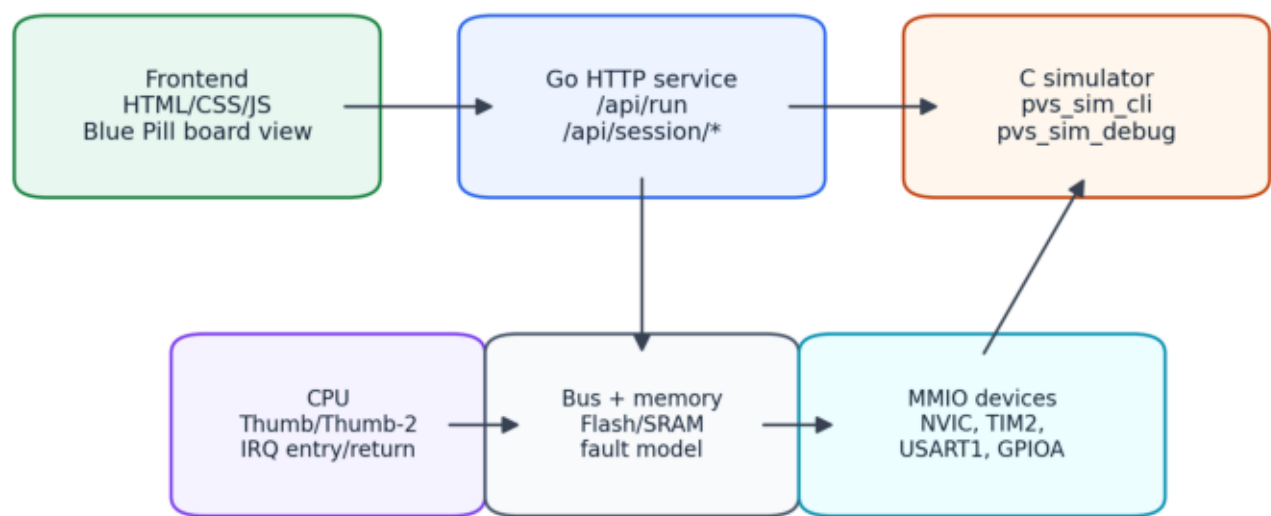
2. Архитектура системы

Система разделена на три слоя. Нижний слой — C-симулятор, где находится состояние CPU, память, шина и устройства. Средний слой — Go-микросервис, отвечающий за HTTP-контракт, запуск subprocess, live sessions и передачу команд. Верхний слой — frontend, предоставляющий пользователю визуальную поверхность управления платой.

Граница ответственности выбрана так, чтобы GUI не зависел от внутренних C-структур и не запускал симулятор напрямую. Все внешние действия проходят через Go API. Это упрощает замену UI, добавление очереди заданий, логирование, трассировку и дальнейшее развитие KeyDB/OpenTelemetry интеграции.

2. Архитектура системы

Архитектура системы



GUI не вызывает C-симулятор напрямую: управление идет через контракт Go-сервиса и stateful debug bridge.

Рисунок 1 — Архитектура системы

2.1 C-симулятор

C-слой содержит ядро симуляции. Структура `sim_t` объединяет CPU, memory, bus, NVIC, TIM2, USART1, GPIOA, UART output buffer и диагностические поля. Выполнение идет по модели fetch-decode-execute с последующим обслуживанием периферии и прерываний.

Шина маршрутизирует обращения CPU в Flash, SRAM и MMIO-регистры. Невыровненные halfword/word-доступы и обращения к неподдержанным MMIO-регистрам завершаются контролируемым fault result, что важно для повторяемого тестирования.

2.2 Go-сервис

Go-сервис предоставляет синхронный endpoint `POST /api/run` и `live/session` endpoints: `create`, `get`, `step`, `run`, `stop`, `events` и `pins`. Для обычного запуска используется `pvs_sim_cli`, для интерактивной отладки — `pvs_sim_debug`, который держит живое состояние `sim_t` между командами.

Команда управления пином проходит через `session.Manager.SetPin` и `debugbridge.Engine.SetPin`.

При наличии `level` сервис отправляет в `debug bridge` строку вида `pin PA0 1`, после чего получает свежий JSON snapshot.

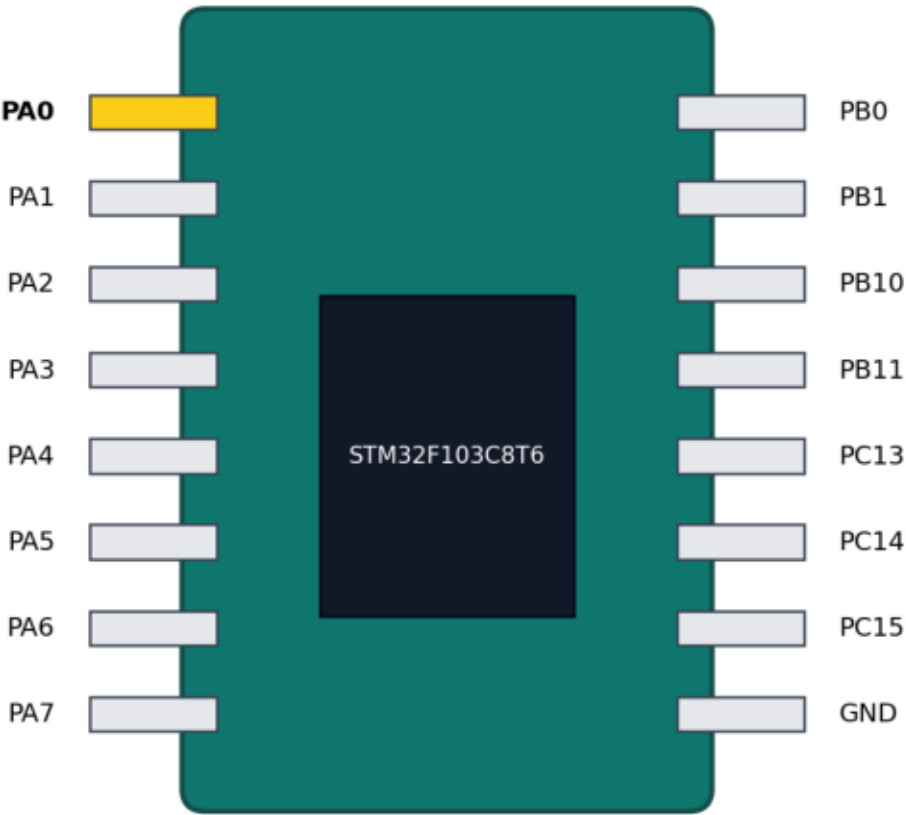
2.3 Frontend

Frontend реализован без npm-зависимостей как статический HTML/CSS/JS. Он умеет создавать live session, выполнять step/run/stop, подписываться на SSE events, показывать UART output, CPU snapshot, peripheral snapshot и визуальное состояние пинов Blue Pill.

В demo-прошивку по умолчанию уже встроен сценарий PA0 -> USART1. Пользователь создает session, кликает PA0 для переключения уровня и запускает прошивку. В UART output появляется H при высоком уровне и L при низком.

2.3 Frontend

Демонстрация Blue Pill: PA0 как управляемый вход



Желтый контакт PA0 переключается из frontend и читается прошивкой через GPIOA->IDR.

Рисунок 2 — Визуальная модель платы и управляемый PA0

3. Описание реализации

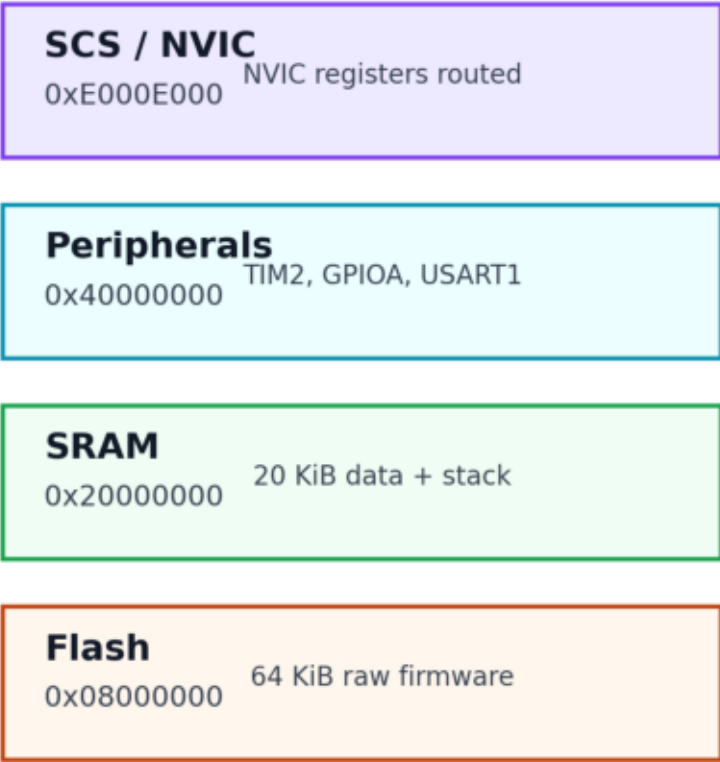
В ходе работы были реализованы CPU L1-задачи, базовая периферия и сервисные контракты. Последний этап сфокусирован на GPIOA и реальном влиянии frontend pin control на исполнение прошивки.

3.1 Минимальная GPIOA/ММО-модель

Добавлен модуль GPIOA с регистрами CRL, CRH, IDR, ODR, BSRR и BRR. CRL/CRH сохраняются для видимости конфигурации, IDR хранит входные уровни, ODR — выходные уровни, BSRR/BRR изменяют выходные биты по правилам STM32-подобной модели.

GPIOA подключен к bus и sim_t. Теперь обращения firmware по адресу 0x40010800 маршрутизируются в gpio_read32/gpio_write32, а debug bridge может обновлять входные уровни через gpio_set_input.

Карта памяти и MMIO



Все обращения CPU проходят через bus; неизвестные MMIO-регистры дают контролируемый fault.

Рисунок 3 — Карта памяти и реализованные MMIO-области

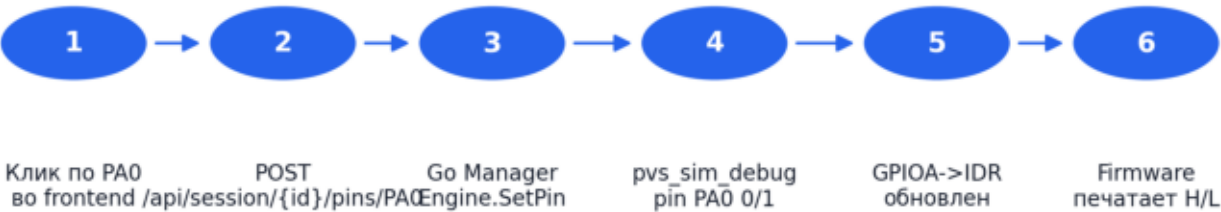
3.2 Stateful debug bridge и управление пинами

В pvs_sim_debug добавлена команда `pin <name> <level>`. Команда поддерживает порт A и уровни 0/1. После применения уровня debug bridge сразу возвращает обновленный JSON snapshot, включая pins array с mode=input и текущим level для PA0-PA7.

В Go-интерфейс Engine добавлен метод `SetPin(ctx, name, request)`. Менеджер сессий перестал ограничиваться session-local overlay: теперь он вызывает `engine.SetPin`, обновляет состояние session и публикует snapshot подписчикам SSE.

3.2 Stateful debug bridge и управление пинами

Сценарий управления пином через Go-сервис



Демонстрационная прошивка читает PA0 через GPIOA IDR и отправляет символ в USART1.

Рисунок 4 — Последовательность управления пином через frontend и Go-сервис

3.3 Демонстрационная firmware-программа

Демонстрационная прошивка инициализирует USART1 TX, читает GPIOA->IDR, маскирует бит PA0 и выбирает символ для передачи. Если PA0 равен 1, в USART1->DR записывается H; если PA0 равен 0, записывается L. Завершение выполняется через SVC как контролируемый break.

Этот сценарий включен в `firmware_integration_test` и в `textarea frontend` по умолчанию. За счет этого отчетный пример воспроизводится без ручной сборки бинарного файла.

4. Тестирование

Тестирование построено по нескольким уровням: сборка CMake, smoke-тесты CPU/peripherals, интеграционные firmware-сценарии, smoke-тест frontend-логики и ручная проверка stateful debug bridge. Такой набор закрывает как нижний C-слой, так и пользовательский путь управления пином.

Таблица 2 — Выполненные проверки

Проверка	Результат
cmake --build build-ninja	Успешно
ctest --test-dir build-ninja --output-on-failure	2/2 tests passed
node gui/tests/app_smoke_test.js	Успешно после запуска вне sandbox
pvs_sim_debug: pin PA0 1 + run	UART output = H, snapshot PA0 level = 1
firmware_integration_test: PA0 low/high	Ожидаемые символы L/H подтверждены

4. Тестирование



Рисунок 5 — Сводка результатов проверок

4.1 CTest

Smoke-тесты проверяют reset sequence, базовые инструкции, флаги, ветвления, работу со стеком, load/store, fault cases, NVIC, TIM2, USART1 и GPIOA MMIO. Интеграционные тесты запускают небольшие прошивки: hello_uart, loop_counter, timer_irq, TIM2 IRQ -> USART1 и PA0 -> USART1.

4.2 Ограничения проверки

В текущем окружении отсутствует `go` в `PATH`, поэтому `go test ./...` не выполнялся. Тем не менее интерфейсные изменения Go-кода были согласованы компиляционно по структуре и проверены через существующие unit-тесты логики там, где доступен Node/CMake. Для полноценного CI рекомендуется добавить среду с Go toolchain.

Заключение

В результате выполненной работы получен рабочий MVP симулятора STM32F103C8T6 с C-ядром, Go-сервисом и статическим frontend. Реализована ключевая вертикаль управления пинами процессора через frontend: UI отправляет запрос в Go-сервис, сервис управляет живой debug session, debug bridge изменяет GPIOA input, а прошивка реагирует на реальный уровень PA0.

Проект готов для демонстрации базовой эмуляции процессора, MMIO-периферии и пользовательского управления входными сигналами. Дальнейшее развитие целесообразно вести в сторону lifecycle cleanup для debug sessions, расширения USART1 RX/IRQ, SysTick/SCB stubs, синхронизации ISA coverage и подключения полноценного Go toolchain в CI.

Список литературы

1. ГОСТ 7.32-2017. СИБИД. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. ГОСТ 2.105-95. ЕСКД. Общие требования к текстовым документам.
3. Калинина М. И., Смирнов С. Б. Методические указания по выполнению выпускных квалификационных работ для бакалавров. Санкт-Петербург: Университет ИТМО, 2016.
4. ARM Architecture Reference Manual ARMv7-M. Architecture profile for microcontrollers.
5. STMicroelectronics. RM0008 Reference manual. STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs.
6. STMicroelectronics. STM32F103C8 datasheet.
7. ПВС 2026 весна. Лекция 1. Практическое задание. PDF-файл, приложенный к проекту.
8. Локальная документация проекта: docs/PROJECT_CONTEXT.md, docs/ARCHITECTURE.md, docs/MMIO_MAP.md, docs/GUI_PLAN.md, docs/TASKS.md.

Приложения

Приложение А. HTTP API live/debug session: POST /api/session создает сессию; POST /api/session/{id}/pins/{name} изменяет входной уровень пина; POST /api/session/{id}/run выполняет прошивку; GET /api/session/{id}/events возвращает SSE stream snapshot-событий.

Приложение Б. Команды проверки: cmake --build build-ninja; ctest --test-dir build-ninja --output-on-failure; node gui/tests/app_smoke_test.js; pvs_sim_debug --firmware <demo.bin> с командами pin PA0 1 и run 20.

Приложение В. Последний коммит реализации: 2dfa609 Wire GPIO pin control through debug sessions. В него вошли GPIOA, SetPin в Go debug bridge, demo firmware, CTest-проверки и обновление документации.